Adaptive Traffic Signal Controller Using IoT, MQTT, Node-RED and Machine Learning Analytics

Project report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by **Team 9**

Samanthula Eswar AM.SC.U4CSE23349

Rayudu Vamsi Krishna AM.SC.U4CSE23344

Bantupalli Harsha Vardhan AM.SC.U4CSE23210

Kota Karthik AM.SC.U4CSE23229

Sammeta Pavan AM.SC.U4CSE23350



AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI CAMPUS

KOLLAM 690525

NOVEMBER 2026

ABSTRACT

Fixed-time traffic signals allocate a preset green duration to each approach, regardless of the traffic waiting at that approach. This can waste green time when demand is low on one approach while queues continue to grow on another. This project, "Adaptive Traffic Signal Controller Using IoT, MQTT, Node-RED and Machine Learning Analytics", develops and evaluates a simulated Internet-of-Things (IoT) adaptive traffic signal controller for a four-approach intersection and compares its behaviour with a fixed-time baseline.

The system connects a Python traffic simulator, an MQTT messaging layer using the Mosquitto broker, Node-RED flows, a SQLite database, a deterministic adaptive controller and a simulated actuator. The simulator publishes traffic readings; Node-RED validates and stores the readings; the controller calculates bounded green durations; and the actuator applies signal commands and returns acknowledgements. A dashboard displays traffic, signal and system information. For portable deployment, the same software is also packaged with Docker and Docker Compose: published container images, a shared Compose configuration, persistent volumes and an optional authenticated Cloudflare Tunnel gateway that exposes only the dashboard. Four machine-learning classifiers are also trained and evaluated for congestion classification. These models are used for offline analytics and evaluation; the live signal controller uses deterministic rules rather than an ML prediction.

The dataset was prepared by Team 9 using real-world traffic data as a reference. It contains 1,250 one-minute timestamps for four approaches, giving 5,000 traffic readings. The team organised the readings, signal-cycle information and adaptive-decision records into a workbook and corresponding CSV files. The original external dataset source details were not available in the project records. Therefore, the dataset is described as team-prepared and derived, not as raw measurements collected by physical roadside sensors. The project also includes a testing environment for actuator validation, fault handling and Stop/Reset behaviour. All reported control results are from simulation and do not demonstrate deployment on a public road. The Docker deployment has been checked for local and tunnelled dashboard access, but a separate clean-laptop run has not yet been independently verified.

Keywords: adaptive traffic signal; Internet of Things; MQTT; Node-RED; SQLite; machine learning; closed-loop control; Docker; containerisation

Table of Contents

Abstract i

List of Figures v

List of Tables vi

List of Abbreviations vii

1 Introduction and Problem Definition 1

1.1 Background and motivation 1

1.2 Problem statement 1

1.3 Objectives 2

1.4 Existing and proposed systems 3

1.5 Scope and assumptions 3

1.6 Report organisation 3

2 System Requirements and Technologies 5

2.1 Technology stack 5

2.2 Functional requirements 6

2.3 Non-functional requirements 7

2.4 Development and evaluation environment 7

3 Dataset Preparation and Methodology 9

3.1 Dataset origin and scope 9

3.2 Dataset structure and files 9

3.3 Dataset attributes and preparation 10

3.4 Feature engineering 10

3.5 Temporal training and test split 10
3.6 Test-set class distribution 11

4 System Architecture and Implementation 12
4.1 Architecture overview 12
4.2 End-to-end workflow 13
4.3 Traffic simulator and message layer 13
4.4 Node-RED and SQLite integration 14
4.5 Deterministic adaptive controller 14
4.6 Actuator and signal-state sequence 14
4.7 Dashboard 14
4.8 Docker-based deployment 15

5 Machine Learning and Computational Evaluation 19
5.1 Role of machine learning in the project 19
5.2 Classification models 19
5.3 Evaluation metrics and classification results 19
5.4 Computational benchmark setup 20
5.5 Computational benchmark results 23

6 Closed-Loop Testing and Safety Verification 26
6.1 Evaluation setup 26
6.2 Evaluation measures 26
6.3 Fixed-time and adaptive simulation results 26
6.4 Actuator acknowledgements and signal safety 28
6.5 Stop and Reset verification 28
6.6 Automated verification 28
6.7 Docker deployment checks 28

7 Results and Discussion 30
7.1 Main findings 30
7.2 Interpretation of traffic-control results 30
7.3 Interpretation of ML results 30
7.4 Interpretation of the Docker deployment 31

7.5 Validity of the evaluation 31

8 Conclusion, Limitations and Future Work **32**

8.1 Conclusion 32

8.2 Limitations 33

8.3 Future work 33

References **35**

A Dashboard and Testing Evidence **36**

List of Figures

4.1 Architecture of the simulated adaptive traffic signal controller. 12

4.2 Project data-preparation, simulation, communication and analytics workflow. The live deterministic controller and actuator-feedback path is shown in Figure 4.1. 13

4.3 Overview of the running traffic dashboard. 15

5.1 Comparison of classification metrics for the four models on the temporal test set. 20

5.2 Confusion matrices for Logistic Regression, Random Forest, Gradient Boosting and MLP on the temporal test set. 21

5.3 Per-class F1-score comparison for the four classifiers. 21

5.4 Comparison of model performance with the majority-class baseline. 22

5.5 Model fitting time measured during the computational benchmark. 23

5.6 Peak process memory (RSS) measured during the computational benchmark. 24

5.7 Median and 95th-percentile inference latency for the four classifiers. 24

5.8 Saved model artifact size for the four classifiers. 25

6.1 Fixed-time and adaptive traffic-control results across delay, queue, fairness and maximum red waiting time in the simulation. 27

A.1 Dashboard view showing signal timing and approach states. 36

A.2 Dashboard view showing traffic analytics. 37

A.3 Testing lab in the running dashboard. 37

A.4 Stop/Reset operation and actuator acknowledgement evidence. 38

A.5 Dashboard evidence of acknowledgement fault handling; an operation with a missing acknowledgement remains unconfirmed. 38

List of Tables

1.1 Objectives and corresponding evaluation evidence 2
1.2 Comparison of fixed-time and proposed adaptive control 3

2.1 Technologies and their roles in the project 5
2.2 Development and evaluation environment 8

3.1 Dataset files and contents 9
3.2 Temporal split used for the ML evaluation 11
3.3 Congestion class distribution in the 1,500-row test set 11

4.1 Configured signal timing parameters 14
4.2 New deployment files added for Docker 16
4.3 Existing project files added or updated for container operation 16
4.4 Published Docker Hub images 17
4.5 Persistent Docker volumes 17

5.1 Machine-learning models evaluated 19
5.2 Classification results on the temporal test set 20
5.3 Approximate computational performance of the four models 22

6.1 Fixed-time versus adaptive traffic simulation results 27
6.2 Stop/Reset acknowledgement verification 28
6.3 Docker deployment checks and security configuration 29

LIST OF ABBREVIATIONS

Abbreviation	**Meaning**
ACK	Acknowledgement
API	Application Programming Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DNS	Domain Name System
F1	F1-score
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JSON	JavaScript Object Notation
ML	Machine Learning
MLP	Multilayer Perceptron
MQTT	Message Queuing Telemetry Transport
QoS	Quality of Service
RAM	Random Access Memory
RSS	Resident Set Size
SQL	Structured Query Language
SQLite	Embedded relational database engine
UI	User Interface

Chapter 1

Introduction and Problem Definition

1.1 Background and motivation

Road traffic in towns and cities is regulated largely by signalised intersections. A signal controller determines which approaches receive the right of way and for how long. Fixed-time controllers repeat a preset cycle of green, yellow and red intervals, often without using the traffic conditions at the intersection at that moment [6]. Such schedules are simple and predictable, but they cannot respond directly to a sudden queue, an empty approach or a change in traffic demand.

The Internet of Things provides a way to connect traffic sensing, communication, data storage and control. Readings can be published by a sensor or simulator, distributed through a messaging broker, processed by software and used to generate a signal command. An acknowledgement from the actuator provides feedback about whether the command was applied. This project implements and evaluates that pipeline for a simulated four-approach intersection using Python, MQTT, Mosquitto, Node-RED and SQLite.

1.2 Problem statement

A fixed-time signal gives each approach a predetermined green duration even when the demand differs between approaches. As a result, a quiet approach may receive unused green time while a busy approach continues to accumulate vehicles. The project addresses the problem of designing and evaluating an IoT-based adaptive controller that reads the traffic state, selects a bounded green duration, applies the signal command through an actuator model and checks the resulting acknowledgement.

The project evaluates whether this adaptive approach improves selected queue and delay measures over a fixed-time schedule on the same prepared traffic demand, while preserving the signal-state sequence and recording command acknowledgements.

1.3 Objectives

The objectives of the project are:

1. Design an IoT pipeline that carries traffic readings from simulation through messaging, storage, analytics, decision and actuation.
2. Prepare a structured dataset containing traffic readings, signal-cycle information and adaptive-decision records for four approaches.
3. Implement a deterministic adaptive controller and a fixed-time baseline for comparison.
4. Train and compare Logistic Regression, Random Forest, Gradient Boosting and MLP classifiers for congestion classification using a temporal data split.
5. Measure model training time, inference latency, memory use and saved model size.
6. Evaluate closed-loop queue behaviour, command acknowledgements and signal safety.
7. Provide a dashboard and testing tools for observing traffic, decisions, actuator state and fault-handling behaviour.
8. Package the complete system with Docker and Docker Compose so that it can be started consistently from published container images, with persistent storage and a restricted, authenticated option for remote dashboard access.

Table 1.1: Objectives and corresponding evaluation evidence

Objective	**Evaluation**	**Evidence**
IoT pipeline	Message flow and data-storage checks	MQTT, Node-RED and SQLite components
Dataset preparation	Row counts, schema and temporal split	Workbook, CSV files and data dictionary
Adaptive controller	Fixed-time versus adaptive simulation	Queue, delay and fairness measures
ML classification	Temporal test-set metrics	Four trained classifiers and result tables
Computational cost	Fit time, latency and memory	Benchmark results
Safety and feedback	Actuator validation and acknowledgement tests	Stop/Reset and fault-handling results
Container deployment	Image publication, Compose startup, and local and tunnelled dashboard checks	Docker files, published images and dashboard/WebSocket check results

1.4 Existing and proposed systems

A fixed-time signal follows a schedule prepared in advance. The green duration does not change in response to the queue currently waiting at an approach. The proposed system uses traffic readings to calculate the next green duration within configured minimum and maximum limits. It also records signal commands and actuator acknowledgements, making the control process observable.

Table 1.2: Comparison of fixed-time and proposed adaptive control

Aspect	**Fixed-time baseline**	**Proposed system**
Green-time decision	Preset duration	Computed from current traffic within configured limits
Response to demand	Does not react during a run	Re-evaluates demand during operation
Feedback	No actuator feedback in the baseline model	Command acknowledgements are recorded
Data handling	Fixed schedule	Traffic readings, decisions and events stored in SQLite
Analytics	No classifier in the baseline	Four offline ML classifiers are evaluated
Testing	Schedule execution	Testing lab, fault checks and Stop/Reset validation

1.5 Scope and assumptions

The implementation covers one simulated intersection with four approaches: North, South, East and West. It includes a traffic simulator, MQTT communication, Node-RED integration, SQLite storage, a deterministic controller, a simulated actuator, an analytics pipeline and a dashboard. It does not include physical roadside sensors or signal heads. Pedestrian phases, emergency-vehicle pre-emption, public-transport priority and coordination between multiple intersections are outside the current scope. The Docker packaging is a deployment layer around the same software simulation; it does not add physical hardware or change the control logic.

The dataset is team-prepared using real-world traffic data as a reference. Since the original external source details were not recorded, it is not treated as a verified raw field dataset. All queue and delay comparisons in this report are simulation results.

1.6 Report organisation

Chapter 2 presents the requirements and technologies. Chapter 3 documents the dataset and experimental methodology. Chapter 4 explains the system architecture and implementation, including the Docker-based deployment. Chapter 5 presents the machine-learning and com-

putational evaluation. Chapter 6 describes closed-loop testing, safety verification and Docker deployment checks. Chapter 7 discusses the results and their validity. Chapter 8 presents the conclusion, limitations and future work.

Chapter 2

System Requirements and Technologies

2.1 Technology stack

The project uses software components that separate traffic simulation, messaging, processing, storage, control and visualisation. Their roles are summarised in Table 2.1.

Table 2.1: Technologies and their roles in the project

Technology	**Role**
Python	Traffic replay, controller logic, actuator simulation, data preparation and ML experiments
MQTT	Publish–subscribe messaging between system components
Eclipse Mosquitto	MQTT broker that receives and distributes messages [2]
Node-RED	Flow-based message ingestion, validation, database integration and dashboard [3]
SQLite	Local storage for readings, decisions, signal events and acknowledgements [4]
Scikit-learn	Training and evaluation of the four classification models [5]
Excel and CSV	Dataset workbook and files used by scripts
Docker	Packages the application, Node-RED and Mosquitto as container images [7]
Docker Compose	Defines and starts the multi-container system, its network, restart policies and volumes [8]
Nginx	Gateway that allows only the authenticated dashboard and WebSocket routes in the optional tunnel setup [9]
Cloudflare Tunnel	Optional temporary (Quick Tunnel) public route to the dashboard gateway [10]

2.1.1 MQTT communication

MQTT is a publish–subscribe messaging protocol. A publisher sends a message to a topic, and the broker forwards it to clients that subscribe to that topic. This separates message producers from consumers. MQTT defines Quality of Service levels that provide different delivery guarantees and overheads [1]. In this project, the broker connects the simulator,

Node-RED, controller and actuator.

2.1.2 Containerisation with Docker

Docker packages an application together with its runtime dependencies into an image that can be run as a container, which reduces differences between development and deployment machines [7]. Docker Compose describes several such containers, their start-up dependencies, networks and storage volumes in one configuration file [8]. In this project, containerisation is used to start the existing simulation consistently; it does not change the controller logic or the system architecture.

2.2 Functional requirements

1. The simulator publishes a traffic reading for each of the four approaches at each simulated time step.
2. The MQTT broker carries traffic readings, signal commands, acknowledgements and status messages.
3. Node-RED validates incoming readings and stores valid records in SQLite.
4. The controller calculates a green duration from current traffic within configured limits.
5. The actuator applies commands using the required signal-state sequence and publishes an acknowledgement.
6. The system stores readings, controller decisions, signal events and acknowledgement information.
7. The dashboard displays traffic and signal information and provides access to testing controls.
8. The analytics pipeline trains and evaluates four congestion classifiers using a temporal split.
9. The system supports a fixed-time baseline for comparison.
10. The Docker deployment starts the controller, publisher, actuator, database initialiser, Node-RED and Mosquitto from container images using Docker Compose.
11. Database history, Node-RED state and broker state persist in named Docker volumes across stop and start.
12. The optional remote-access setup exposes only the authenticated dashboard page and its WebSocket route.

2.3 Non-functional requirements

- **Safety:** the actuator must prevent conflicting green states and enforce clearance intervals.
- **Reproducibility:** dataset versions, temporal split boundaries and experiment settings should be recorded.
- **Observability:** commands, events and acknowledgements should be timestamped and stored.
- **Modularity:** components should have clearly defined responsibilities and exchange data through configured interfaces.
- **Efficiency:** the system and benchmarked models should run on a CPU-only development machine.
- **Validity:** simulation findings and the team-prepared dataset must not be represented as physical field measurements.
- **Portability:** the system should be startable from published images and a Compose file, with container-specific settings kept separate from the Windows settings.
- **Security:** public access must be authenticated and limited to the dashboard; the MQTT port, the SQLite database and the Node-RED editor must not be exposed publicly, and credentials must be kept out of Git, images and shared reports.

2.4 Development and evaluation environment

The recorded benchmark environment was Windows 11 with Python 3.13.7, 12 logical CPU cores, scikit-learn 1.7.2 and NumPy 2.2.6. The ML experiments were run on CPU without a GPU. The IoT pipeline uses Eclipse Mosquitto, Node-RED and SQLite. The project is a software simulation; no physical traffic-signal hardware is claimed. The Docker deployment uses Docker Compose with three published images (Section 4.8); the machine-learning benchmark results in Chapter 5 were recorded in the environment above and were not repeated inside containers.

Table 2.2: Development and evaluation environment

Item	**Recorded environment or role**
Operating system	Windows 11
Python	3.13.7
CPU	12 logical CPUs; CPU-only evaluation
NumPy	2.2.6
Scikit-learn	1.7.2
Messaging	Eclipse Mosquitto with MQTT
Flow integration	Node-RED
Database	SQLite
Container deployment	Docker Compose with three published images
Optional public access	Nginx gateway with Cloudflare Quick Tunnel

Chapter 3

Dataset Preparation and Methodology

3.1 Dataset origin and scope

The dataset used in this project was prepared by Team 9 using real-world traffic data as a reference. The team organised and derived traffic readings, signal-cycle information and adaptive-decision records into a structured Excel workbook and corresponding CSV files for simulation, analysis and evaluation. The original external dataset source details were not available in the project records.

Accordingly, the 5,000 traffic readings are described as a team-prepared dataset. They are not described as raw measurements collected directly from physical roadside sensors. No unsupported external source name or URL is added to this report.

3.2 Dataset structure and files

The master workbook is named `adaptive_traffic_project_dataset.xlsx`. The dataset is organised into traffic readings, signal-cycle information, adaptive decisions and fixed-time baseline information. The related files are listed in Table 3.1.

Table 3.1: Dataset files and contents

File	Content
`adaptive_traffic_project_dataset.xlsx`	Master workbook containing the linked dataset sheets
`traffic_data.csv`	Traffic readings for the four approaches; 5,000 rows
`signal_cycle.csv`	Signal-cycle and phase information
`adaptive_decisions.csv`	Adaptive green-time decision records
`fixed_baseline_signal.csv`	Fixed-time baseline information
`data_dictionary.md`	Column definitions and data-origin description

The time axis contains 1,250 one-minute timestamps from 03:00 to 23:49 on 12 October 2026. Each timestamp has one row for each of the four approaches, giving $1{,}250 \times 4 =$

5,000 traffic readings. The approaches are North, South, East and West. The workbook keeps traffic readings, signal-cycle records, adaptive decisions and the fixed-time baseline in separate linked structures.

3.3 Dataset attributes and preparation

The traffic records describe the timestamp, approach and traffic conditions such as queue or vehicle count, flow, speed and occupancy. Identifiers such as `Cycle_No`, `Decision_ID` and `Event_ID` connect relevant records across the dataset files. The detailed column definitions are maintained in `data_dictionary.md`.

The preparation process used the following steps:

1. Establish a one-minute time axis containing 1,250 timestamps.
2. Represent each of the four approaches at every timestamp.
3. Organise the traffic readings and related signal-cycle and decision records into linked sheets.
4. Keep the fixed-time baseline information separate from adaptive-decision records.
5. Export the workbook sheets to CSV files for use by Python scripts.
6. Maintain a data dictionary describing the columns and the derived nature of the dataset.

Signal states GREEN, YELLOW and ALL_RED are generated by the simulator and actuator logic; they are not treated as raw traffic sensor fields in the master traffic dataset.

3.4 Feature engineering

The ML pipeline converts the traffic readings into model features. The feature set includes current queue and flow values, lagged values, short rolling averages, changes from the previous minute and time-of-day indicators. Features are calculated within each approach and use past observations only, so that future readings are not used to construct an earlier feature row. Where scaling is required, the scaler is fitted on the training period only.

3.5 Temporal training and test split

The data are split chronologically rather than randomly. A temporal split is more appropriate for this experiment because a deployed controller would make decisions about future traffic using information available up to the current time. A random split could place neighbouring observations in both training and test subsets and produce an overly optimistic estimate.

The dataset contains 5,000 source traffic readings. Feature engineering produces 4,996 feature rows. The final temporal split contains 3,492 training rows and 1,500 test rows; four boundary rows are excluded. The latest training target timestamp is 17:33 on 12 October 2026, and the earliest test feature timestamp is 17:34 on the same date.

Table 3.2: Temporal split used for the ML evaluation

Subset	**Rows**	**Notes**
Original traffic readings	5,000	Four approaches across 1,250 timestamps
Feature rows generated	4,996	After temporal feature construction
Training set	3,492	Earlier observations
Test set	1,500	Later observations
Boundary rows excluded	4	Excluded at the temporal split boundary

3.6 Test-set class distribution

The test set contains three congestion categories: Low, Medium and High. The class distribution is imbalanced, with the Low category forming the largest share. This is why accuracy is considered together with balanced accuracy, macro F1-score and High-class recall.

Table 3.3: Congestion class distribution in the 1,500-row test set

Congestion category	**Rows**	**Share**
Low	1,017	67.80%
Medium	385	25.67%
High	98	6.53%
Total	1,500	100.00%

Chapter 4

System Architecture and Implementation

4.1 Architecture overview

The system is organised as connected components. The Python simulator represents the traffic sensing layer and publishes readings. Mosquitto distributes MQTT messages. Node-RED validates incoming data, writes records to SQLite and supplies information to the dashboard. The deterministic controller processes current traffic information and publishes signal commands. The simulated actuator applies the command, enforces the signal sequence and publishes an acknowledgement.

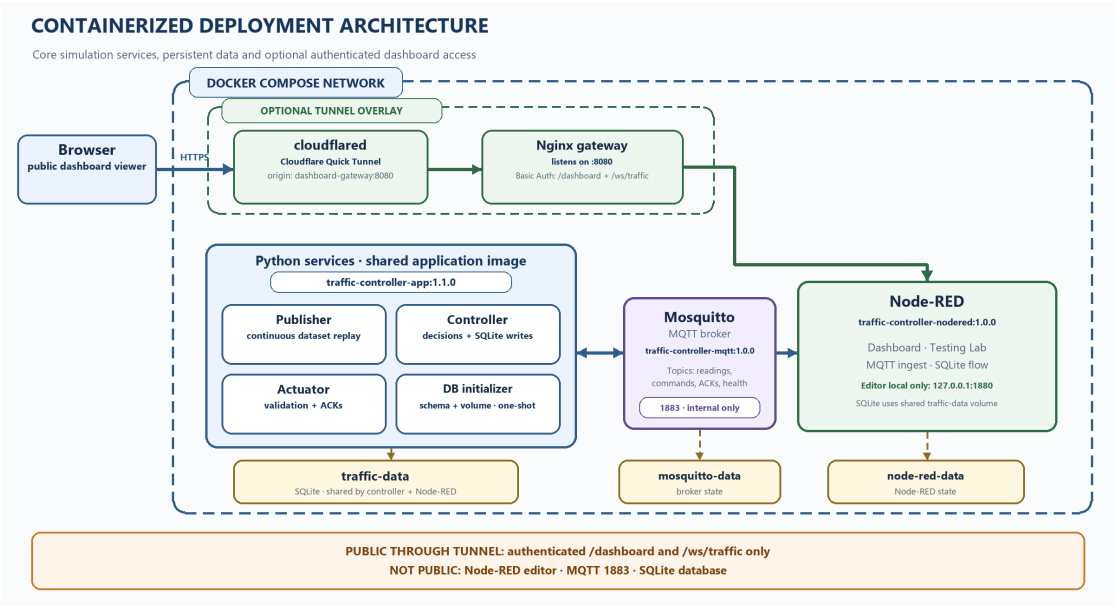


Figure 4.1: Architecture of the simulated adaptive traffic signal controller.

4.2 End-to-end workflow

The live control sequence is described below. The workflow illustration summarizes the data-preparation, simulation, communication and analytics workstream; it is not intended to place offline ML inference in the live signal-control path.

1. The Python simulator replays one traffic reading per approach per simulated minute.
2. Mosquitto distributes the readings to the subscribed processing components.
3. Node-RED validates the incoming records and stores valid readings in SQLite.
4. The dashboard displays traffic, current signal states, decisions and acknowledgements.
5. The deterministic controller calculates the green duration for the next phase within configured limits.
6. The controller publishes the signal command to the simulated actuator.
7. The actuator applies the command using the required signal-state sequence.
8. The actuator publishes an acknowledgement, which is recorded for feedback and evaluation.

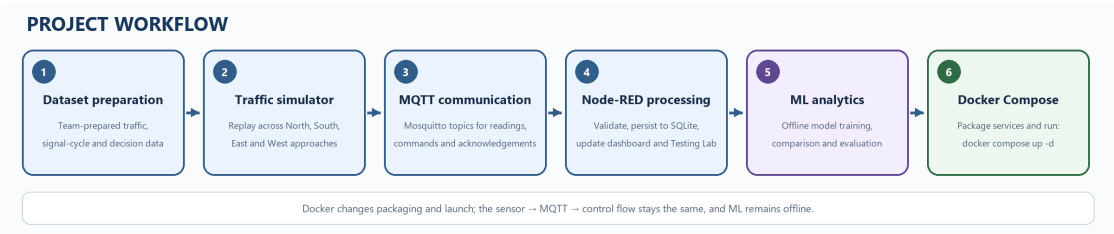


Figure 4.2: Project data-preparation, simulation, communication and analytics workflow. The live deterministic controller and actuator-feedback path is shown in Figure 4.1.

4.3 Traffic simulator and message layer

The simulator replays the prepared traffic readings as if they were arriving from roadside detectors. It publishes the traffic state of each approach and supports accelerated replay for testing. It also supplies the simulated intersection state. The MQTT layer separates message producers from consumers: the simulator publishes readings, the controller publishes commands, and the actuator publishes acknowledgements and status information.

The message layer uses separate topic branches for readings, commands, acknowledgements and status. Messages are structured as JSON and carry timestamps or identifiers needed to associate readings with decisions and signal events. Exact live topic names should be

taken from the current project configuration when preparing any supplementary topic listing; illustrative topic names are not substituted for verified implementation values in this report.

4.4 Node-RED and SQLite integration

Node-RED subscribes to relevant MQTT messages, checks that required fields are present and that values are within configured ranges, and writes valid records to SQLite. Invalid data are handled separately rather than being treated as valid traffic readings. The flows also provide dashboard updates and expose the project's testing controls.

SQLite stores traffic readings, controller decisions, signal events and acknowledgement information. This provides a persistent record for the dashboard, test review and evaluation scripts.

4.5 Deterministic adaptive controller

The live controller uses deterministic rules based on current traffic information. It calculates green durations subject to configured minimum and maximum limits. This bounds the time assigned to any one approach and prevents a phase from being held indefinitely. A fixed-time baseline uses a constant green duration for comparison.

Table 4.1: Configured signal timing parameters

Parameter	**Configured value**
Minimum green time	10 seconds
Default green time	25 seconds
Maximum green time	60 seconds
Yellow duration	3 seconds
All-red duration	2 seconds

4.6 Actuator and signal-state sequence

The actuator receives a command identifying the phase and requested green duration. It applies signal changes through the required GREEN, YELLOW and ALL_RED sequence. It rejects unsafe commands, including commands that would result in conflicting greens or skip a required clearance interval. The actuator returns an acknowledgement describing the command outcome. The acknowledgement is used to verify that the command was processed rather than assuming that publication alone proves successful actuation.

4.7 Dashboard

The Node-RED dashboard provides a visual view of the simulated intersection. It presents traffic readings, queues, signal states, controller decisions, acknowledgements and system

status. The primary dashboard route in the project is `/dashboard`; the additional `/ui/` route is secondary. The Docker deployment serves the same Node-RED dashboard, and its appearance is the same as in the original setup. Figure 4.3 and the screenshots in Appendix A therefore represent both the original and the Docker deployments.

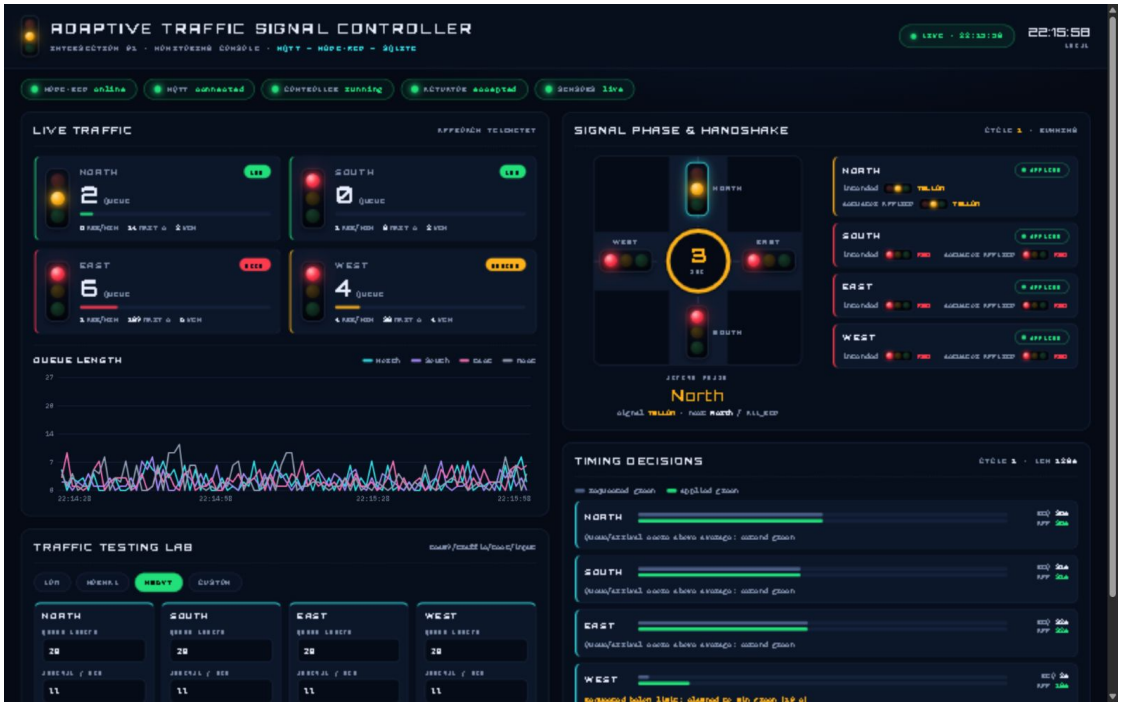


Figure 4.3: Overview of the running traffic dashboard.

4.8 Docker-based deployment

4.8.1 Baseline project and what Docker added

Before the Docker work, the project was already a complete software simulation of a four-approach intersection. It contained the Python traffic-replay and simulation code, the deterministic adaptive controller with configured safe timings, the simulated actuator with command validation and acknowledgements, MQTT messaging through Mosquitto, Node-RED flows for validation, dashboard updates, Testing Lab controls and SQLite persistence, the dataset and ML artifacts, analytics and evaluation results, automated tests, Windows startup scripts and a development `requirements.txt`. The historical file guide recorded this base project as 79 files at the time of its inventory on 15 October 2026. Docker was added as a packaging and launch layer so that the same workflow can be started consistently; the traffic-control behaviour itself was not redesigned. Docker packages the components shown in Figure 4.1 without changing that architecture.

Table 4.2: New deployment files added for Docker

File	Purpose
`Dockerfile`	Builds one Python application image shared by the controller, publisher, actuator and database initialiser
`docker-compose.yml`	Defines the services, dependencies, restart policies, internal network and persistent named volumes
`docker-compose.tunnel.yml`	Optional local overlay adding the Cloudflare Tunnel and dashboard gateway
`tunnel/nginx.conf`	Nginx gateway that allows only the authenticated dashboard and WebSocket routes; Docker DNS resolution follows Node-RED if its container IP changes
`mqtt/Dockerfile`, `mqtt/mosquitto.conf`	Mosquitto image setup and broker configuration
`node_red/Dockerfile`	Node-RED image with the dashboard, project flow and SQLite support
`requirements-docker.txt`	Minimal Python runtime dependencies for the application image
`.dockerignore`	Keeps local tunnel credentials and unrelated files out of the application image build context
`README-Docker.md`	Docker setup, security, persistence, Quick Tunnel and another-laptop instructions

Table 4.3: Existing project files added or updated for container operation

File	Change
`config/docker-settings.yaml`	Container-specific settings, including the Compose MQTT hostname and continuous replay behaviour; the Windows settings remain separate
`src/mqtt/publisher.py`	Supports live timestamps during replay so that historical dataset timestamps do not make incoming readings appear stale
`.gitignore`	Excludes the local dashboard password, its authentication hash and local database data
`README.md`, `PROJECT_FILE_GUIDE.md`	Updated to describe the Docker deployment and image names

4.8.2 Container images and services

Three images are published on Docker Hub (Table 4.4). The Python services share the application image, while Node-RED and Mosquitto use their own images.

Table 4.4: Published Docker Hub images

Image	Used for
`eswarsamanthula/traffic-controller-app:1.1.0`	Controller, publisher, actuator and database initialiser
`eswarsamanthula/traffic-controller-nodered:1.0.0`	Node-RED flows and dashboard
`eswarsamanthula/traffic-controller-mqtt:1.0.0`	Mosquitto MQTT broker

The Compose file starts these services on an internal Docker network, with start-up dependencies and restart policies. Three named volumes keep state outside the containers (Table 4.5). A new laptop starts with fresh volumes and therefore with empty database history.

Table 4.5: Persistent Docker volumes

Volume	Contents
`traffic-data`	SQLite history
`node-red-data`	Node-RED state
`mosquitto-data`	Mosquitto broker state

4.8.3 Starting and stopping the system

From a folder containing `docker-compose.yml`, the portable stack is started with:

```
docker compose pull
docker compose up -d
```

The optional local Cloudflare Quick Tunnel is started with the tunnel overlay:

```
docker compose -f docker-compose.yml -f docker-compose.tunnel.yml up -d
docker compose -f docker-compose.yml -f docker-compose.tunnel.yml logs -f cloudflared
```

The `cloudflared` log prints a temporary `trycloudflare.com` address; the dashboard is reached by appending `/dashboard`. This address changes after `cloudflared` stops. All services are stopped, with volumes and database history preserved, using:

```
docker compose -f docker-compose.yml -f docker-compose.tunnel.yml stop
```

4.8.4 Security of the Docker deployment

The public tunnel is authenticated and, through the Nginx gateway, allows only the dashboard page and its WebSocket route. The Node-RED editor is not passed through the gateway. MQTT port 1883 and the SQLite database are not exposed publicly. The dashboard password

is kept in a local file that is excluded from Git, from the application image build context and from this report; the password and its authentication hash must not be committed, baked into an image or shared. The deployment remains a software simulation.

Chapter 5

Machine Learning and Computational Evaluation

5.1 Role of machine learning in the project

The machine-learning component classifies congestion levels from engineered traffic features. The models are trained and assessed offline to compare predictive performance and computational cost. They are not the live signal-timing decision-maker in the current implementation. The live controller uses deterministic rules with configured timing bounds.

5.2 Classification models

Four models are evaluated: Logistic Regression, Random Forest, Gradient Boosting and a Multilayer Perceptron (MLP). The MLP is a feed-forward neural network with two hidden layers of 64 and 32 units.

Table 5.1: Machine-learning models evaluated

Model	**Description**
Logistic Regression	Linear classification baseline
Random Forest	Ensemble of decision trees
Gradient Boosting	Sequential ensemble of decision trees
MLP	Feed-forward neural network with hidden layers of 64 and 32 units

5.3 Evaluation metrics and classification results

The classification models are compared using accuracy, balanced accuracy, macro F1-score and High-class recall. Accuracy reports the proportion of all test examples classified correctly. Balanced accuracy gives equal importance to recall across classes. Macro F1 averages the class-level F1-scores, and High-class recall measures the proportion of genuinely

High-congestion examples detected by the model.

Table 5.2: Classification results on the temporal test set

Model	**Accuracy**	**Balanced accuracy**	**Macro F1**	**High recall**
Logistic Regression	0.6987	0.4238	0.4384	0.1327
Random Forest	0.6713	0.4538	0.4747	0.2143
Gradient Boosting	0.7067	0.4163	0.4258	0.0714
MLP	0.6773	0.4815	0.5014	0.2857

The MLP gives the highest macro F1-score and High-class recall among the four models, while Gradient Boosting gives the highest overall accuracy. The test set's majority-class baseline accuracy is 0.6780. The MLP's accuracy of 0.6773 is slightly below that baseline, which shows why accuracy alone is not sufficient to assess the models. High-class recall remains low for every model, so the results do not support describing the classifier as a reliable detector of every high-congestion event.

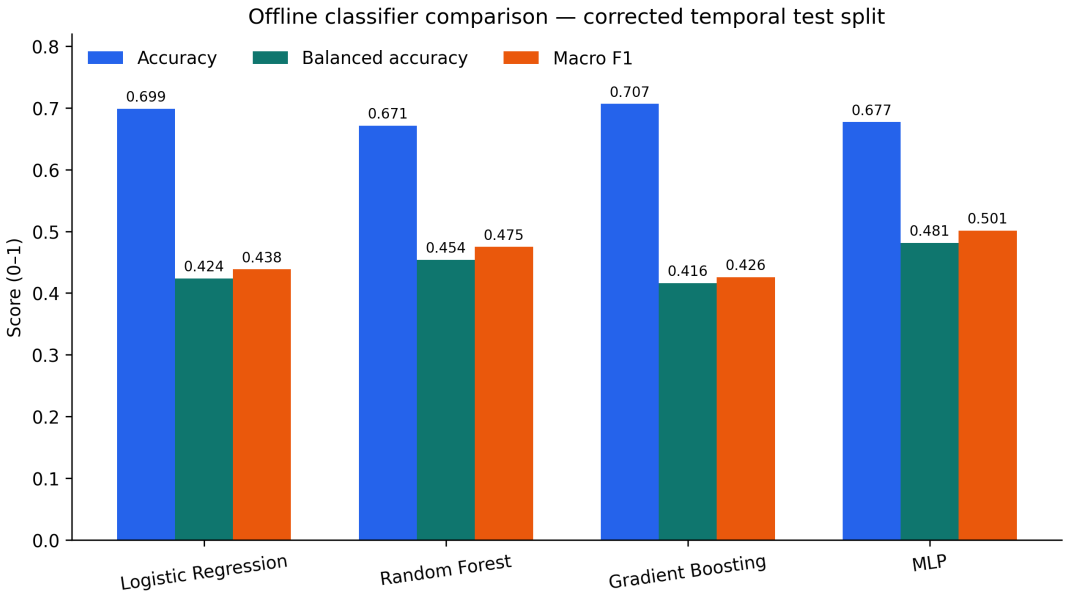


Figure 5.1: Comparison of classification metrics for the four models on the temporal test set.

5.4 Computational benchmark setup

The computational benchmark was run on Windows 11 using Python 3.13.7, 12 logical CPUs, scikit-learn 1.7.2 and NumPy 2.2.6. The experiment was CPU-only. Three fit runs were performed for each model. The recorded quantities include mean fit time, peak process memory (RSS), median and 95th-percentile inference latency, and saved artifact size.

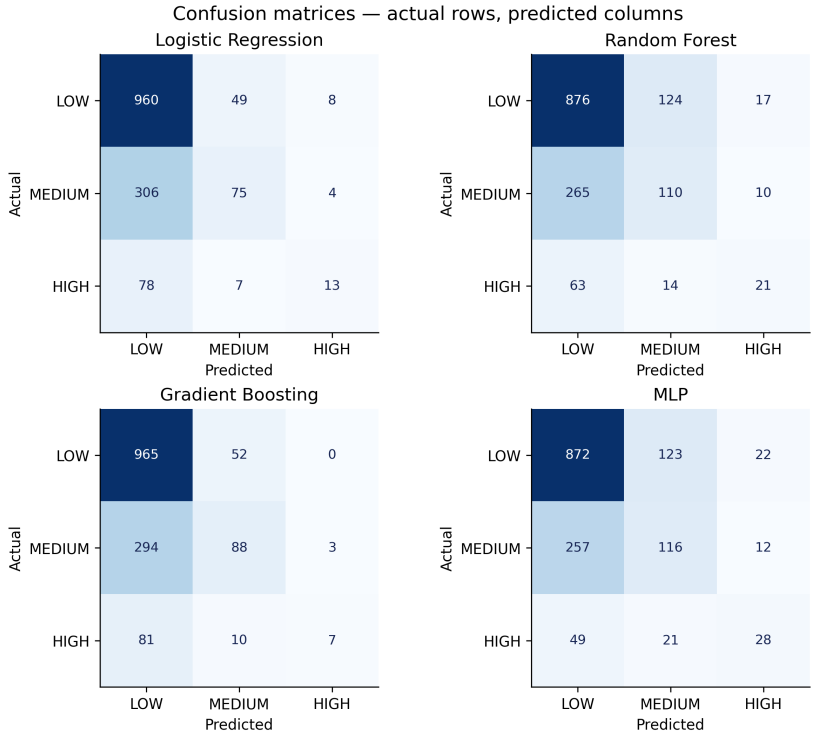


Figure 5.2: Confusion matrices for Logistic Regression, Random Forest, Gradient Boosting and MLP on the temporal test set.

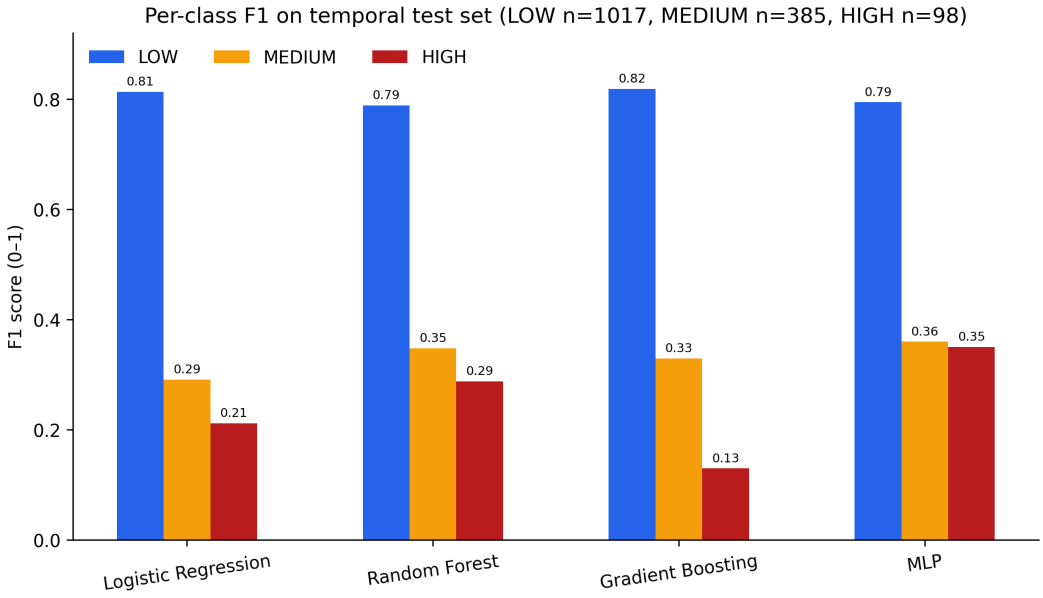


Figure 5.3: Per-class F1-score comparison for the four classifiers.

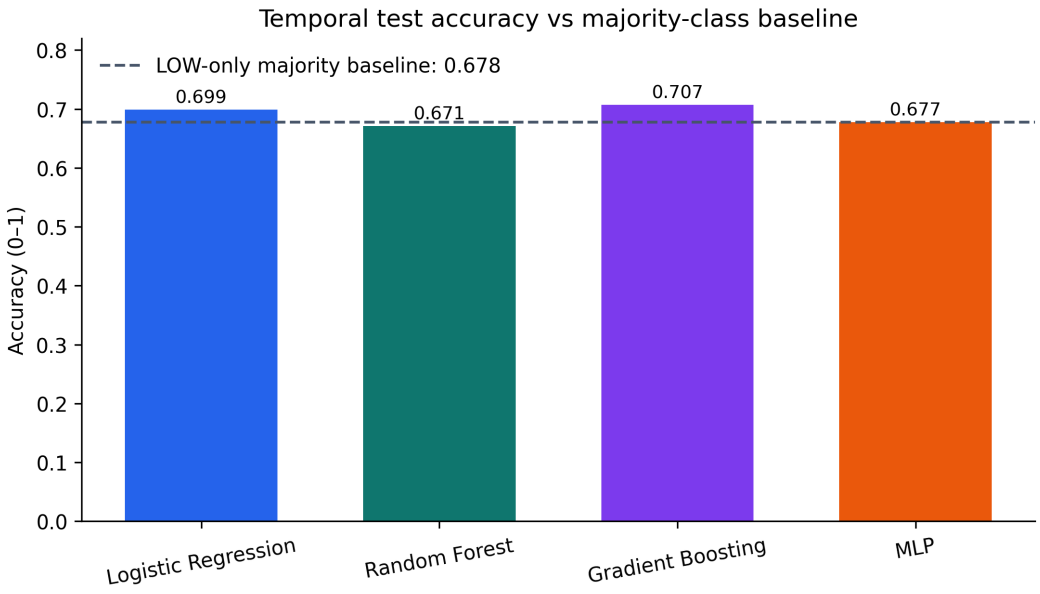


Figure 5.4: Comparison of model performance with the majority-class baseline.

Table 5.3: Approximate computational performance of the four models

Model	**Fit (s)**	**Peak RSS (MB)**	**p50 (ms)**	**p95 (ms)**	**Artifact (MB)**
Logistic Regression	0.0253	153.9	0.094	0.191	0.030
Random Forest	0.7969	249.5	4.912	6.806	53.972
Gradient Boosting	0.7152	270.7	0.342	0.611	0.361
MLP	2.3105	273.5	0.110	0.128	0.129

5.5 Computational benchmark results

The reported benchmark values are specific to the development machine and benchmark procedure. Logistic Regression has the shortest fit time and smallest saved artifact. Random Forest has the largest artifact and highest reported p50/p95 inference latency among these models. MLP has the highest macro F1-score and High-class recall, but it does not have the highest overall accuracy. Model choice therefore depends on the metric being prioritised; these offline measurements do not mean that any of the four models controls the live signal phases.

The available benchmark summary also contains throughput figures, but it does not specify the prediction batch size or denominator clearly enough to compare them directly with single-record p50/p95 latency. Those figures are not reproduced here without the original benchmark definition.

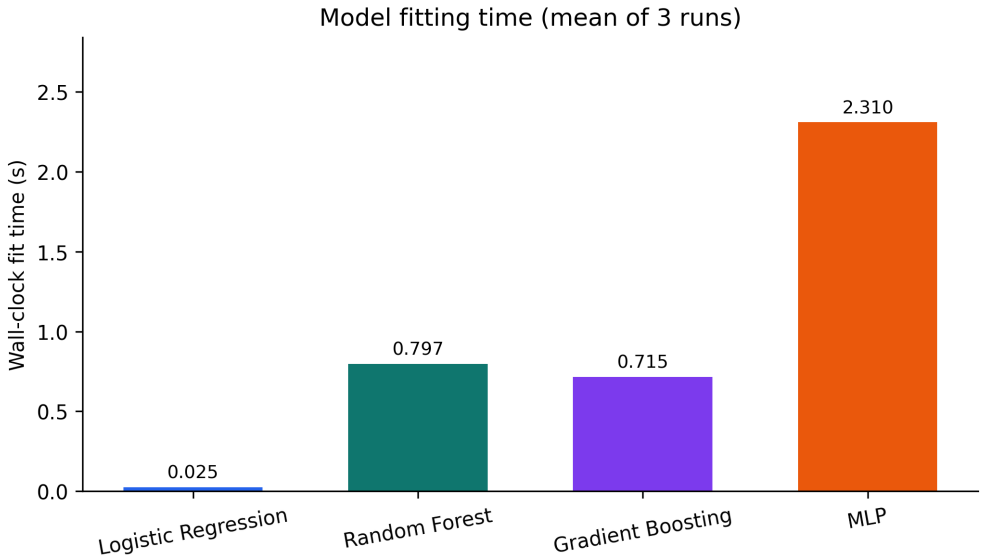


Figure 5.5: Model fitting time measured during the computational benchmark.

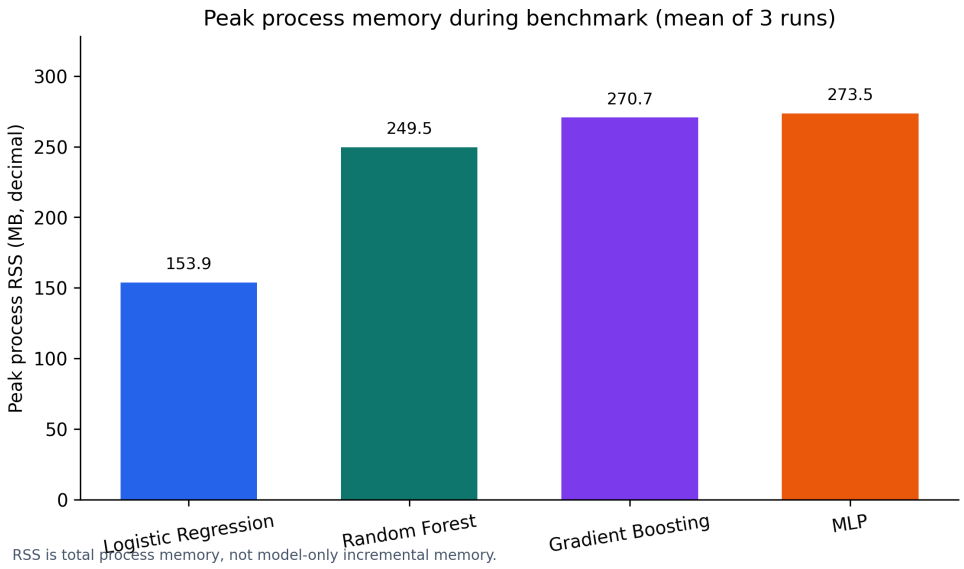


Figure 5.6: Peak process memory (RSS) measured during the computational benchmark.

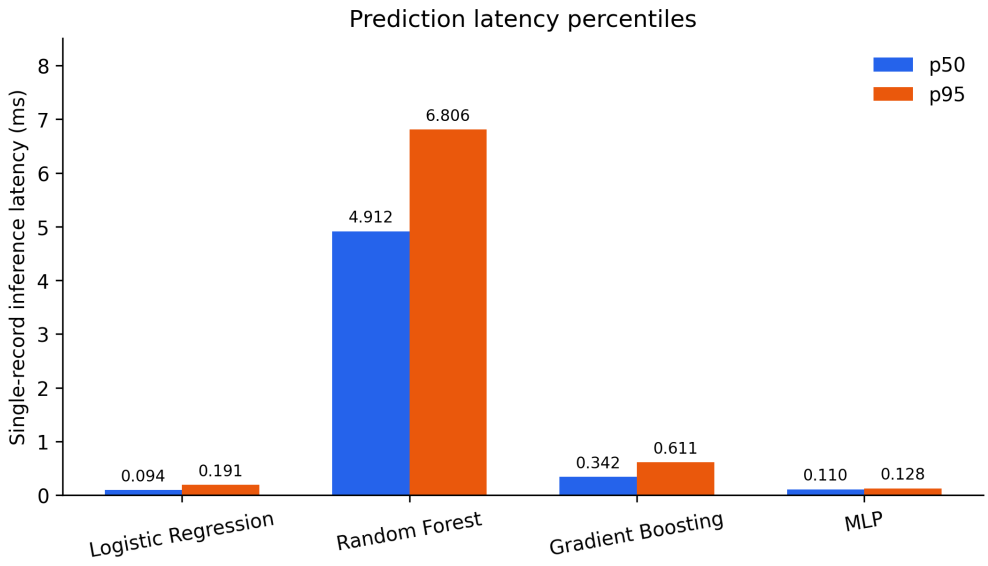


Figure 5.7: Median and 95th-percentile inference latency for the four classifiers.

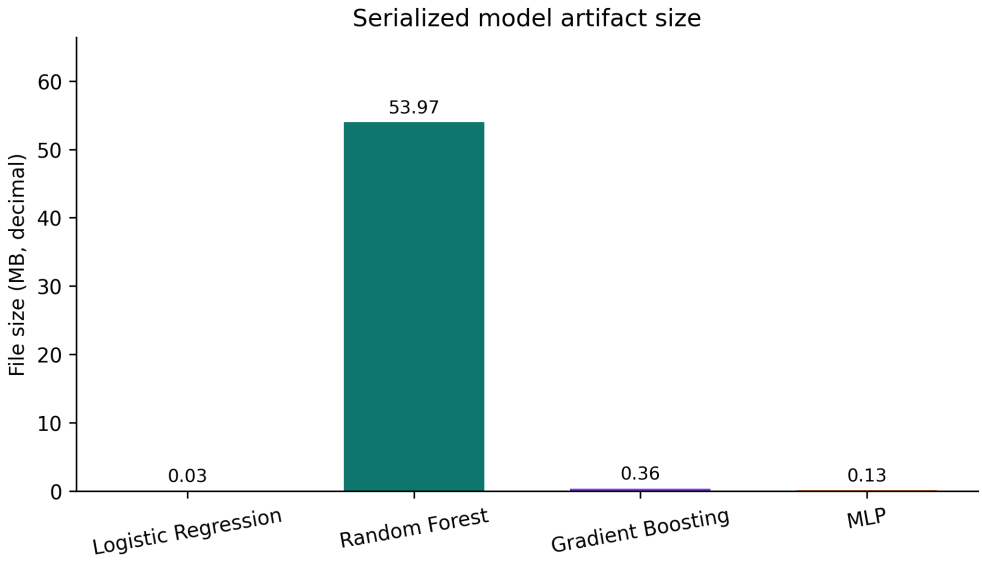


Figure 5.8: Saved model artifact size for the four classifiers.

Chapter 6

Closed-Loop Testing and Safety Verification

6.1 Evaluation setup

The closed-loop checks exercise the controller and actuator using an isolated in-process test environment. The setup includes the actual controller logic, controller service, actuator node or simulator, a validation component and a temporary SQLite database. A manual clock is used to make timing behaviour repeatable. The resulting latency measurements represent application-level processing in this test harness; they do not include delay from a physical MQTT network or a deployed broker connection.

The fixed-time and adaptive simulations use the same prepared traffic demand so that the comparison focuses on the difference between the scheduling strategies. The evaluation is still a simplified software simulation rather than a measurement at a physical intersection.

6.2 Evaluation measures

The closed-loop evaluation records average delay, average queue, maximum queue, fairness and maximum red waiting time. It also checks the number of actuator events, whether commands receive acknowledgements and whether the signal-state sequence is respected.

6.3 Fixed-time and adaptive simulation results

The adaptive run has lower average delay and queue and a lower maximum queue in this simulation. Its fairness index is also higher. However, the maximum red waiting time rises from 95 seconds to 164 seconds. This is an important trade-off: better average results do not guarantee a better worst-case wait for every approach or vehicle. The maximum red waiting time must remain visible in the evaluation rather than being omitted from the result summary.

Table 6.1: Fixed-time versus adaptive traffic simulation results

Measure	**Fixed-time**	**Adaptive**
Average delay (seconds)	139.399	76.585
Average queue	7.005	3.848
Maximum queue	113	21
Jain's fairness index	0.9572	0.9923
Maximum red waiting time (seconds)	95	164

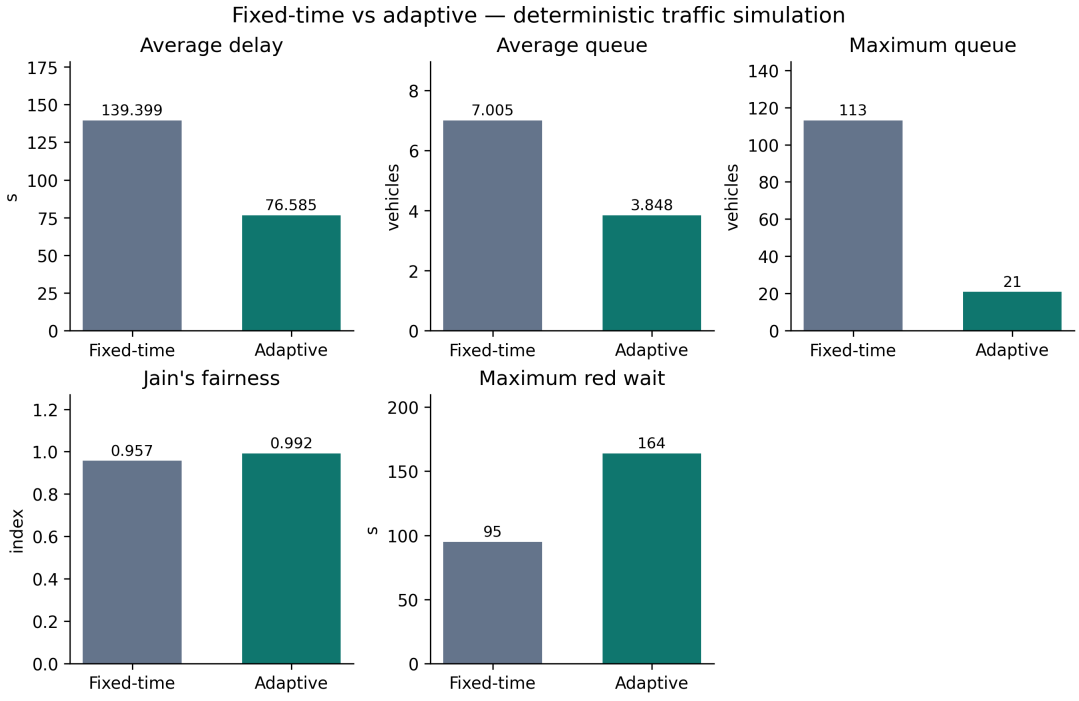


Figure 6.1: Fixed-time and adaptive traffic-control results across delay, queue, fairness and maximum red waiting time in the simulation.

6.4 Actuator acknowledgements and signal safety

The actuator validation checks that signal commands follow the allowed GREEN, YELLOW and ALL_RED sequence and that conflicting approaches are not granted green simultaneously. The testing harness records commands and acknowledgements so that an issued command is not automatically counted as a successfully applied command.

The normal closed-loop test recorded 37 phase events and received 37 acknowledgements. The successful path had no reported safety violations in the validation run. These observations apply to the simulated actuator and test harness only.

6.5 Stop and Reset verification

The Stop and Reset procedure sends a distinct RED command for each of the four approaches. The operation is confirmed only after all four commands receive accepted acknowledgements. A successful Stop or Reset therefore requires 4/4 acknowledgements. In the fault-injected case where an acknowledgement is withheld, the operation remains unconfirmed and restart is inhibited. This failure case is intentional evidence that the system does not report a successful safe-state transition without the required acknowledgement.

Table 6.2: Stop/Reset acknowledgement verification

Test condition	**Observed behaviour**	**Outcome**
All four RED commands acknowledged	4/4 accepted acknowledgements received	Operation confirmed
One acknowledgement withheld	Operation remains unconfirmed; restart inhibited	Failure handled safely

6.6 Automated verification

The final reported automated test run contained 200 passing tests and zero failures. The successful safety paths reported no conflicting-green or skipped-clearance violations. The acknowledgement-withheld test remains a negative test: it is expected to prevent confirmation and restart rather than be reported as a successful Stop/Reset operation.

6.7 Docker deployment checks

The Docker deployment was checked at the dashboard and gateway level. With authentication, local and public dashboard requests returned HTTP 200, and the authenticated WebSocket returned the current healthy actuator status. These checks show that the containerised dashboard path and the authenticated tunnel route were working in the tested run.

These checks do not replace the closed-loop and safety tests above, and they do not show

Table 6.3: Docker deployment checks and security configuration

Item	**Result or configuration**
Local dashboard request (authenticated)	HTTP 200
Public tunnel dashboard request (authenticated)	HTTP 200
Authenticated WebSocket	Returned current healthy actuator status
Node-RED editor	Not passed through the gateway
MQTT port 1883 and SQLite database	Not exposed publicly
Run on a separate clean laptop	Not yet independently verified

that the stack has been run on a different machine.

Chapter 7

Results and Discussion

7.1 Main findings

The evaluation produces three main findings. First, the adaptive controller improves the average queue and delay measures in the prepared simulation when compared with the fixed-time baseline. Second, the machine-learning results depend on the metric: Gradient Boosting has the highest accuracy, while MLP has the highest macro F1-score and High-class recall. Third, the actuator and Stop/Reset checks show the importance of acknowledgements: a safe-state operation is not confirmed when one of the required acknowledgements is missing.

7.2 Interpretation of traffic-control results

In the recorded fixed-time versus adaptive run, average delay decreases from 139.399 seconds to 76.585 seconds (about 45.1% lower), and average queue decreases from 7.005 to 3.848 (about 45.1% lower). Maximum queue decreases from 113 to 21 (about 81.4% lower). Jain's fairness index increases from 0.9572 to 0.9923.

The maximum red waiting time, however, increases from 95 seconds to 164 seconds (about 72.6% higher) in the adaptive run. Therefore, the adaptive schedule improves several aggregate measures while worsening this worst-case waiting measure. Further controller tuning should target this trade-off, for example by incorporating an explicit maximum-wait constraint and then re-evaluating both average and worst-case performance.

7.3 Interpretation of ML results

The congestion test set is imbalanced, with 67.80% of its rows labelled Low. The majority-class accuracy baseline is 0.6780. Logistic Regression and Gradient Boosting exceed that accuracy baseline, while Random Forest and MLP do not. MLP nevertheless produces the best macro F1-score and High-class recall among the four models. These values show that

the models differ in how they trade overall accuracy against performance on minority classes. High-class recall is low for every model, so further work is required before a congestion classifier could be relied upon for safety-related decisions.

The live controller does not currently use these ML predictions. It uses deterministic rules, which keeps the current signal-timing behaviour separate from the offline ML experiment and avoids overstating the role of machine learning in the implemented system.

7.4 Interpretation of the Docker deployment

Docker addresses a practical problem of the original project: the system depends on Python, Mosquitto, Node-RED and SQLite, which previously had to be prepared and started separately. Packaging them as published images with a Compose file allows the stack to be started with `docker compose pull` and `docker compose up -d`, and named volumes keep database, Node-RED and broker state across stop and start. Because the dashboard served from the containers looks the same as before, the existing dashboard figures apply to both deployments. The optional tunnel adds remote viewing, but only for the authenticated dashboard and WebSocket routes, so the editor, MQTT port and database stay private. Docker changes how the simulation is deployed and accessed; it does not change the controller, the simulated results or the ML findings.

7.5 Validity of the evaluation

The traffic dataset was prepared by the team using real-world traffic data as a reference; the original external source details are not available in the project records. The traffic-control comparison is a simplified simulation on this prepared demand, and the actuator is simulated. The in-process closed-loop test measures application-level processing and does not include physical network or field-device latency. Consequently, the results are useful for comparing the implemented software paths under the defined test conditions, but they are not evidence of measured improvement on a public road.

The ML test set comes from a temporal split of a single prepared day. Performance may differ on other days, intersections, traffic distributions or sensor types. Computational results also depend on the development machine and measurement method. For the Docker deployment, local and tunnelled dashboard checks were successful, but a separate clean-laptop run has not been independently verified. Portability is therefore prepared through published images and Compose, and cross-laptop execution should not be described as tested.

Chapter 8

Conclusion, Limitations and Future Work

8.1 Conclusion

This project implemented a simulated IoT-based adaptive traffic signal controller for a four-approach intersection. The software pipeline connects a Python traffic simulator, MQTT communication through Mosquitto, Node-RED integration, SQLite storage, a deterministic controller, a simulated actuator and a dashboard. The actuator returns acknowledgements, allowing the system to verify command outcomes rather than assuming that a published command has been applied.

The team prepared a structured dataset with 5,000 traffic readings across 1,250 one-minute timestamps and four approaches. Four ML classifiers were evaluated using a chronological split. The ML results show that the best model depends on the metric, and the current classifiers have limited High-class recall. The live controller remains rule-based. In the recorded traffic simulation, the adaptive controller reduced average delay and queue and improved the fairness index, but it also increased the maximum red waiting time. Stop/Reset validation confirmed successful operation only after all four RED acknowledgements were received and prevented restart when an acknowledgement was withheld.

The system was also packaged with Docker and Docker Compose. Three images are published on Docker Hub, named volumes preserve database, Node-RED and broker state, and an optional authenticated Cloudflare Tunnel with an Nginx gateway exposes only the dashboard and its WebSocket route. Local and public dashboard checks returned HTTP 200 and the authenticated WebSocket returned healthy actuator status. The deployment remains a software simulation, and its execution on a separate clean laptop has not yet been independently verified.

8.2 Limitations

- The dataset is team-prepared using real-world traffic data as a reference. Its original source details were not recorded, and it is not a collection of verified raw roadside measurements.
- The system is a software simulation of one four-approach intersection; no physical sensors, signal heads or real road deployment were tested.
- The traffic-control comparison is based on a simplified prepared demand and does not represent every condition at a real intersection.
- The maximum red waiting time increased in the reported adaptive run, despite improvements in average queue and delay.
- The ML experiment uses one temporal test period from a single prepared day, and the High class is under-represented.
- The in-process closed-loop latency test excludes physical MQTT network, broker and field-device delays.
- Pedestrian phases, emergency-vehicle pre-emption, public-transport priority and multi-intersection coordination are not implemented.
- The Docker deployment has not been independently run on a separate clean laptop, so cross-laptop portability is prepared but not verified.
- The Cloudflare Quick Tunnel address is temporary and changes after `cloudflared` stops, and a new Docker deployment starts with empty database history.

8.3 Future work

- Collect or obtain a documented multi-day traffic dataset and validate the controller against traceable field measurements.
- Add more intersection layouts and evaluate the controller across different traffic patterns, weather and peak periods.
- Introduce explicit maximum-wait and fairness constraints to reduce the worst-case red waiting time.
- Add pedestrian phases, emergency-vehicle pre-emption and public-transport priority.
- Evaluate coordination between multiple intersections along a corridor.

- Investigate whether ML congestion forecasts can safely support the deterministic controller, with a separate evaluation before enabling any live influence.
- Add MQTT authentication and encryption and test the system on embedded hardware before any real deployment.
- Include physical network and actuator timing in a future end-to-end performance evaluation.
- Run and document the Docker deployment on a clean laptop to verify portability, and consider a stable authenticated address in place of the temporary Quick Tunnel URL if long-term remote access is needed.

References

[1] OASIS, "MQTT Version 5.0," OASIS Standard, March 2019.

[2] R. A. Light, "Mosquitto: server and client implementation of the MQTT protocol," *Journal of Open Source Software*, vol. 2, no. 13, p. 265, 2017.

[3] OpenJS Foundation, "Node-RED: Low-code programming for event-driven applications," project documentation, `https://nodered.org`.

[4] SQLite Consortium, "SQLite documentation," `https://www.sqlite.org/docs.html`.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] F. V. Webster, "Traffic signal settings," Road Research Technical Paper No. 39, Her Majesty's Stationery Office, London, 1958.

[7] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.

[8] Docker Inc., "Docker Compose documentation," `https://docs.docker.com/compose/`.

[9] Nginx, "nginx documentation," `https://nginx.org/en/docs/`.

[10] Cloudflare, "Cloudflare Tunnel," Cloudflare documentation, `https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/`.

Appendix A

Dashboard and Testing Evidence

This appendix contains genuine screenshots captured from the running project dashboard. All image files are uploaded to the same root folder as this `IOT_Report.tex` file in Overleaf; therefore, figure commands use filenames without a `report_assets/dashboard/` directory prefix. The Docker deployment serves the same dashboard, so these screenshots are used for both the original and the Docker deployments.

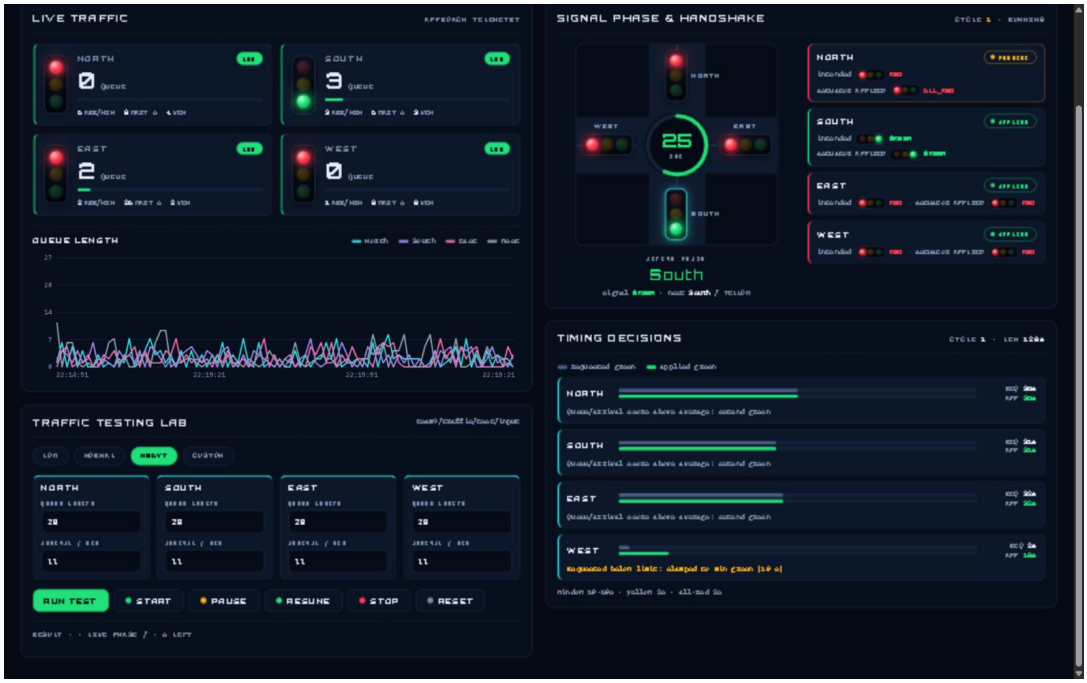


Figure A.1: Dashboard view showing signal timing and approach states.

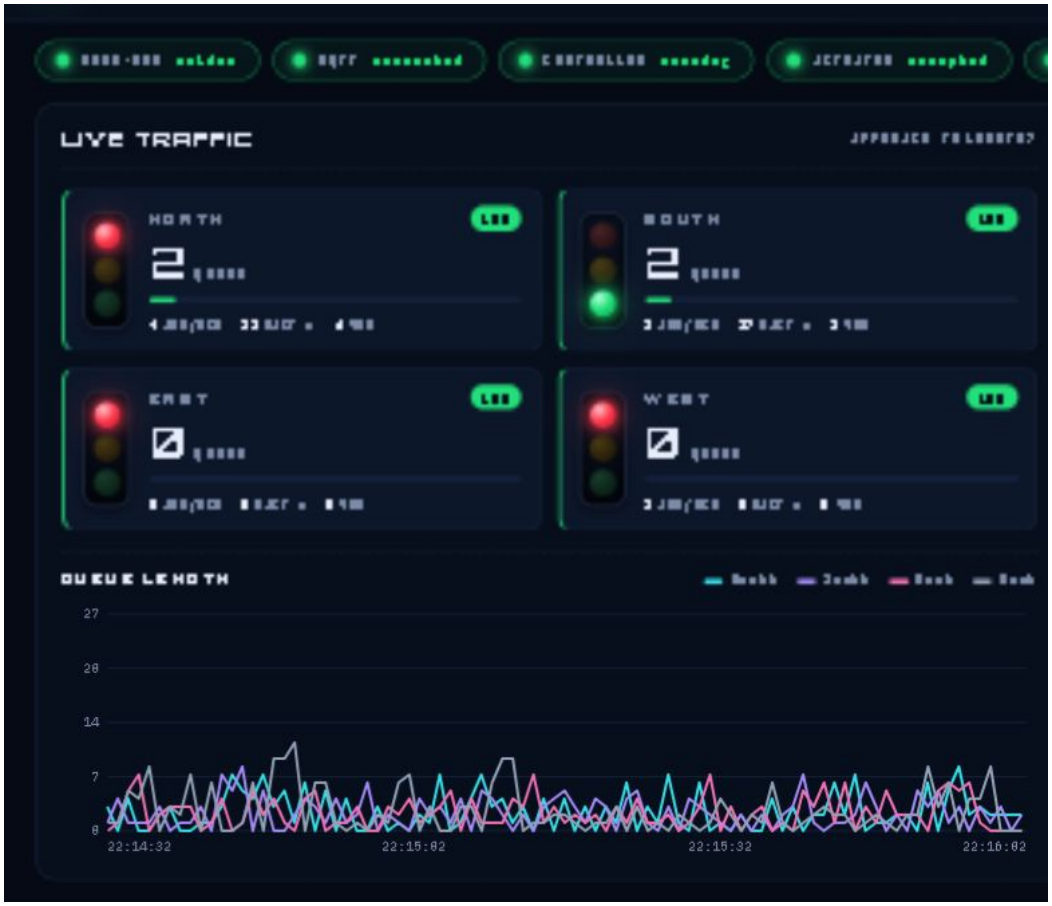


Figure A.2: Dashboard view showing traffic analytics.

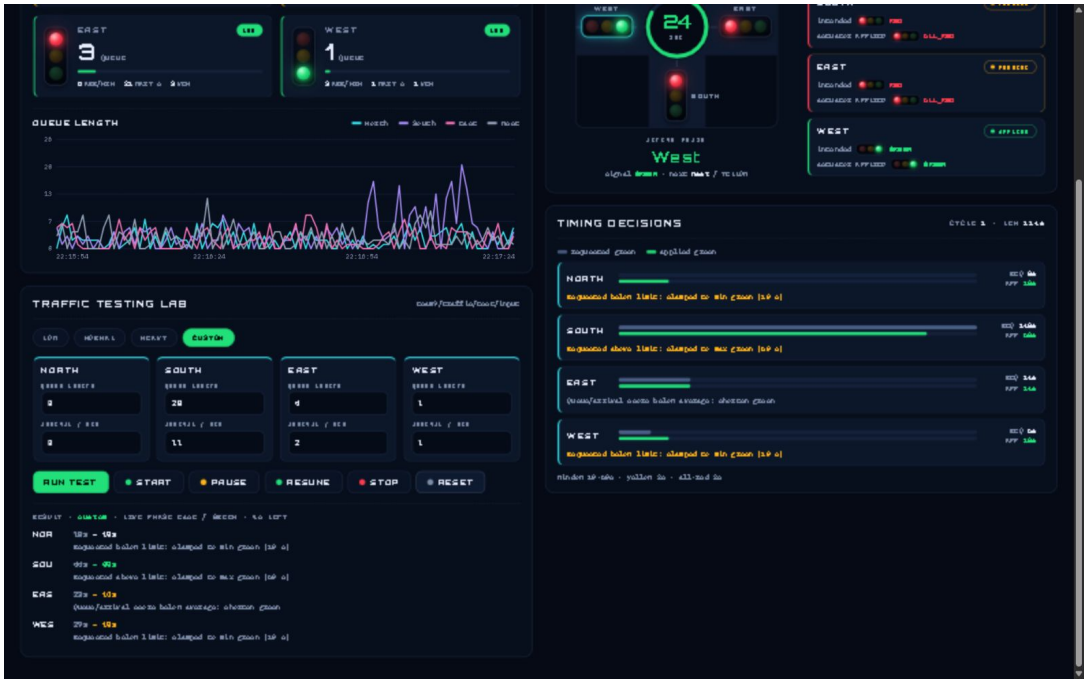


Figure A.3: Testing lab in the running dashboard.

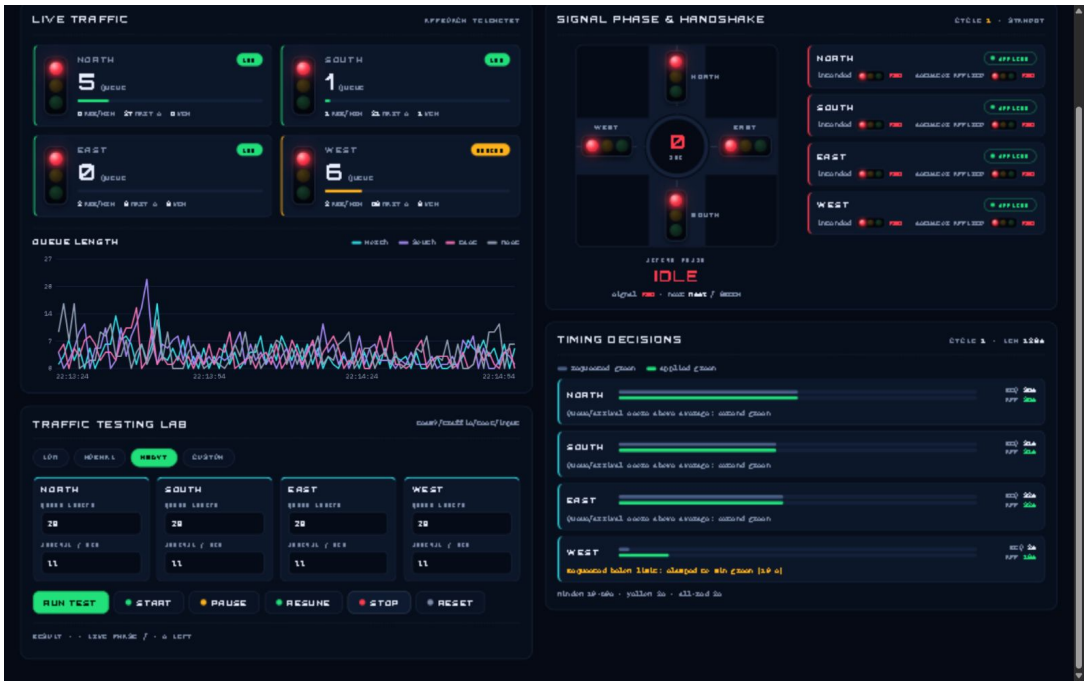


Figure A.4: Stop/Reset operation and actuator acknowledgement evidence.

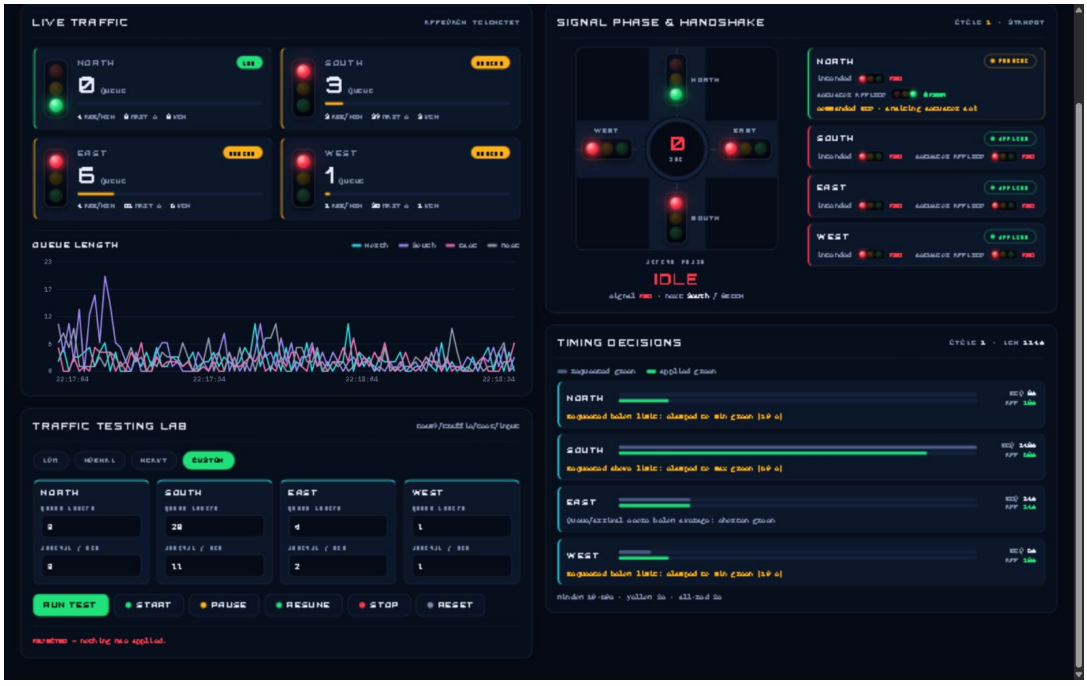


Figure A.5: Dashboard evidence of acknowledgement fault handling; an operation with a missing acknowledgement remains unconfirmed.